

B Prompt Templates

B.1 Knowledge Probe Prompt

Source function: `get_knowledge_probe_prompt(...)`

You are a medical prior-knowledge assistant. Generate clinical prior knowledge for the given
 ↪ tabular medical features.
 Write EVERYTHING in English only.
 [Optional if {task_description} is non-empty]
 Task description: {task_description}
 Outcome/target column: {target}
 You MUST return a Markdown table that includes the relationships between the features and the
 ↪ outcome, with exactly these columns:
 | Feature | Univariate signal (summary) | Clinical rationale | Suggested threshold | Evidence
 ↪ confidence (high/medium/low) |
 Requirements:
 - You must provide a suggested threshold (if not applicable, write "no clear threshold" and explain
 ↪ why)
 - Clinical rationale should be specific enough to justify rules
 - Evidence confidence must be one of: high / medium / low
 Feature list (json):
 {features_json}

B.2 Initial Rule Generation Prompt

Source function: `get_rule_generation_prompt(...)`

You are a medical rule-learning agent. Based on the input information, you will generate a
 ↪ pure-Python classification rule function.
 Write EVERYTHING in English only.
 Inputs include: univariate summary, medical knowledge table, and metric priority.
 [Optional if {task_description} is non-empty]
 Task description: {task_description}
 {metric_desc}

[Univariate Summary]
 {univariate_summary}

[Medical Knowledge Table]
 {knowledge_table}

Return STRICT JSON with fields:
 - version: "v0"
 - error_analysis: the design rationale (in English)
 - new_policy_code: full Python function definition; function name MUST be predict_v0
 Function signature MUST be: `def predict_v0(features: dict) -> int:`
 Return an integer class label. For binary tasks, return 0/1 matching the dataset label.
 You may implement a score-based rule, a direct rule-based decision tree, or any deterministic rule
 ↪ set, as long as it is a pure-Python function.
 Each if/elif/else branch MUST include an English comment briefly explaining the medical rationale
 ↪ or design intent.
 The rule must be self-contained and use ONLY the Python standard library (no third-party packages).

B.3 Iteration Prompt

Source function: `get_iteration_prompt(...)`

You are a medical rule-learning agent. Update the current Python-based classification rule based
 ↪ on:
 - Current full code (all historical versions)
 - This round's training-set error analysis
 - Iteration trajectory (reasons for previous changes)
 - Metric priority

- (If any) degradation warning: cases that regressed from correct to wrong after the last change

[Optional if {task_description} is non-empty]
Task description: {task_description}

{metric_desc}

[Current Full Code]
{current_code}

[Training Error Analysis]
{error_report}

[Iteration Trajectory]
{trajectory}

[Degradation Warning]
{degradation_warning}

Important: if a degradation warning exists, you MUST prioritize fixing regressed cases (previously
→ correct, now wrong) and try not to introduce new regressions.

Each if/elif/else branch MUST include an English comment briefly explaining the medical rationale
→ or design intent.

Extra constraint: do NOT collapse to predicting almost all 1s or 0s.

Minimal-change constraint: only make small adjustments this round (e.g., adjust 1-2

→ thresholds/weights, or add/remove no more than 2 rules). Do NOT rewrite the whole function.

Return STRICT JSON:

```
{
  "version": "{next_version}",
  "error_analysis": "...(English)...",
  "new_policy_code": "def predict_...\n ..."
}
```

Keep changes minimal and comments clear.

The rule must be self-contained and use ONLY the Python standard library (no third-party packages).

B.4 Continual Learning v0 Generation Prompt

Source function: get_continuous_v0_generation_prompt(...)

You are a medical rule-learning agent. I have already built a pure-Python classification rule

→ function. However, feature drift has occurred. Based on the following summary of feature drift,

→ together with the provided univariate summary, medical knowledge table, metric priority, and

→ other inputs, update the classification rule function.

Write EVERYTHING in English only.

Inputs include: feature drift summary, updated univariate summary, updated medical knowledge table,

→ metric priority, and the previous final model blueprint.

[Optional if {task_description} is non-empty]

Task description: {task_description}

{metric_desc}

[Feature Drift Summary]

- Dropped columns: {dropped_cols}

- Added columns: {added_cols}

- Renamed columns: {renamed_cols}

- Change note: {change_note}

[Optional if {univariate_summary} is non-empty]

[Updated Univariate Summary]

{univariate_summary}

[Optional if {knowledge_table} is non-empty]

[Updated Medical Knowledge Table]

{knowledge_table}

[Previous Final Model Blueprint]

{blueprint_code}

Return STRICT JSON with fields:

- version: "v0"
- error_analysis: the design rationale (in English)
- new_policy_code: full Python function definition; function name MUST be predict_v0

Function signature MUST be: def predict_v0(features: dict) -> int:

Return an integer class label. For binary tasks, return 0/1 matching the dataset label.

Do not reference dropped features.

You may use added or renamed features if useful.

Each if/elif/else branch MUST include an English comment briefly explaining the medical rationale

↔ or design intent.

The rule must be self-contained and use ONLY the Python standard library (no third-party packages).